Semester project on control and simulation of automation systems

BEng in Robot Systems

4. Semester

Authors (Group 1)

Name	Username	Date of birth
Noah V. Vryens	novry24	01/04/2002
Rasmus K. Nielsen	rasni19	25/12/1997
Emma B. Rasmussen	erasm24	15/05/2001
Jannick H. Irvold	jairv24	23/10/2002
Eymundur Ó. Pálsson	eypal24	31/08/2002

Supervisor

Cheng Fang

chfa@mmmi.sdu.dk

Total characters:

55.389

May 28, 2026

In this project, the goal was to balance an inverted pendulum attached to a cart, driven by a belt. To achieve stable balancing of the pendulum, mathematical models of the system were developed using both the Newton-Euler and Euler-Lagrange methods. The resulting dynamic models were linearized around the upright equilibrium position and used for simulation and controller development in MATLAB/Python.

In addition to simulation, the project also focuses on implementation aspects related to real automation systems. This includes communication between software and hardware, PLC-based control implementation, and validation of the simulated models using the physical setup and tests. The project therefore demonstrates both the theoretical and practical challenges associated with modelling and controlling unstable dynamic systems in modern automation engineering.

The modeling, controller design and implemented resulted in three controllers using different strategies, which all managed to stabilize the inverted pendulum. Though the controllers were stable, the physical performance did not fully match simulated results, likely due to unmodelled physical effects such as static friction, backlash, sensor noise, filtering delays actuator saturation and small mechanical imperfections in the setup.

Contents

1 Introduction

Control systems play an essential role in modern automation and robotics, where precise and stable motion is required in order to achieve reliable system behavior. From industrial robots and autonomous vehicles to balancing systems and advanced manufacturing equipment, controllers are widely used to regulate unstable and dynamic systems.

One of the most well-known and challenging control problems is the inverted pendulum system. The system is naturally unstable, meaning even small disturbances can cause the pendulum to fall if no corrective action is applied. Due to its nonlinear and unstable behavior, the inverted pendulum has become a classical benchmark problem within control engineering and is frequently used for research, education, and validation of different control strategies.

The purpose of this project is to model, simulate, and control an inverted pendulum mounted on a conveyor-driven cart system. The project combines several engineering disciplines, including dynamic modelling, simulation, control theory, hardware implementation, and automation systems integration.

1.1 Project Goals

Using a conveyor-driven cart system, inverted pendulum, and different controller designs, the objective of this project is to stabilize the inverted pendulum in its upright position by regulating the force applied by the motor. The performance of each controller is evaluated by comparing the behavior of the physical system with the corresponding simulation model, in addition to comparing the performance parameters between the controllers. The controller with the response most closely matching the simulated behavior and target performance requirements while maintaining stable pendulum balancing is considered the most successful controller implementation.

1.2 Problem Definition

The main objectives of this project are to investigate and implement different control strategies capable of stabilizing an inverted pendulum mounted on a conveyor-driven cart system. The project focuses on both modelling and practical controller implementation.

- Model the dynamic behavior of the inverted pendulum system.
- Implement classical and modern control strategies.
- Simulate and validate the controllers in MATLAB/Python.
- Implement the controllers on a physical system.
- Compare the responses of the different control strategies.

1.3 Constraints & Limitations

The project is subject to several physical, technical, and implementation-related limitations that influence both the modelling process and the achievable control performance of the system.

One of the primary physical limitations comes from the conveyor-driven cart system. The cart movement is constrained by the length of the conveyor rail and the placement of the sensors on the sides, limiting the maximum allowable travel distance during operation. This restricts how aggressively the controller can react before the cart reaches the physical boundaries of the system.

Additionally, the motor and conveyor system are limited by the maximum available actuation force and mechanical response of the hardware. These limitations affect the achievable acceleration and response time of the cart, which directly influences the ability to stabilize the pendulum under dynamic conditions.

The physical setup also introduces uncertainties and non-ideal effects that are difficult to model perfectly. These include friction in the conveyor system, damping in the pendulum joint, mechanical vibrations, sensor noise, backlash, and small structural imperfections in the setup. In the mathematical model, many of these effects are simplified using linear damping coefficients.

From a control perspective, the system represents a nonlinear and inherently unstable problem. To simplify the controller design, the system dynamics were linearized around the upright equilibrium position using small-angle approximations. This means that the mathematical model is primarily valid for small pendulum angles close to the vertical operating point. Large disturbances or extreme pendulum angles may therefore result in deviations between the simulated and physical system behavior.

The hardware implementation also introduces computational and communication-related limitations. The controller execution frequency, encoder resolution, and communication delays between the hardware and external software systems affect the responsiveness and stability of the implemented controller. Additionally, communication delays introduced through OPC UA may affect the responsiveness and real-time performance of the implemented control system compared to the ideal simulation environment.

1.4 System Overview

The physical setup consists of several interacting subsystems, including:

- A conveyor-driven cart system using a Schneider Electric 6000 rpm AC servomotor [1], along with a Lexium 32C Servo Drive [2].
- An inverted metal pendulum with a small tip mass, attached at the center of the cart.
- Encoders placed at the pendulum attachment point and on the servomotor.
- A B&R Programmable Logic Controller (PLC), X20CP1382 [3].

2 System Modeling

2.1 Modelling of the Inverted Pendulum System

The purpose of the modelling process is to obtain a mathematical representation of the inverted pendulum system suitable for simulation and controller design. A mathematical model makes it possible to analyze the dynamic behavior of the system and evaluate different control strategies before implementation on the physical setup.

In this project, two classical modelling approaches were used: the Newton-Euler method and the Euler-Lagrange method. Both methods describe the dynamics of the system from different physical perspectives. The Newton-Euler method is based on force and moment balances, while the Euler-Lagrange method is based on the kinetic and potential energy of the system.

Both modelling approaches resulted in coupled nonlinear differential equations describing the dynamic behavior of the system. To simplify the controller design and simulation process, the nonlinear equations were linearized around the upright equilibrium position using small-angle approximations.

The resulting linearized models were transformed into transfer functions and state-space representations suitable for simulation, controller development, and implementation in MATLAB/Python.

The generalized coordinates used to describe the system, as seen in figure 2.1 are:

- x: horizontal cart position
- θ: pendulum angle relative to the vertical axis

The physical parameters used in the model are shown in Table 2.1.

Table 2.1: System parameters used for modelling of the inverted pendulum system [4]

Symbol	Parameter	Value	Unit
M	Mass of cart	0.500	kg
m	Mass of pendulum (including tip mass)	0.084	kg
l	Length of pendulum rod	0.35	m
L_{belt}	Length of conveyor belt	1.72	m
r	Radius of belt rollers	0.05	m
g	Gravitational acceleration	9.82	m/s^2
b_{lin}	Linear damping coefficient	5	N/(m/s)
b_{rot}	Rotational damping coefficient	0.0012	Nm/(rad/s)

The following assumptions were made during the modelling process:

- The pendulum rod is considered rigid
- Friction is assumed to be small and primarily represented by damping coefficients
- The pendulum motion is constrained to a two-dimensional plane
- The system is linearized around the upright equilibrium position

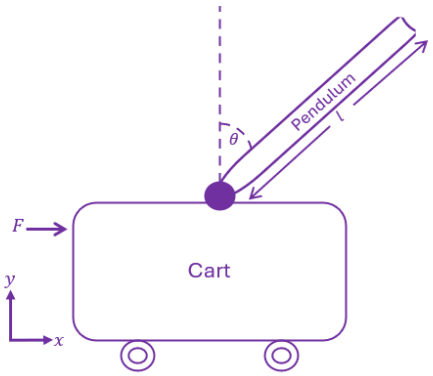

Figure 2.1: Simplified model of the inverted pendulum system showing the generalized coordinates, pendulum angle, and input force.

2.1.1 Newton-Euler Modelling

The Newton-Euler method is based on applying Newton's second law and rotational moment equations to the system. A free body diagram, as seen in figure 2.2, was constructed for both the cart and pendulum to identify the forces and moments acting on the system. [5]

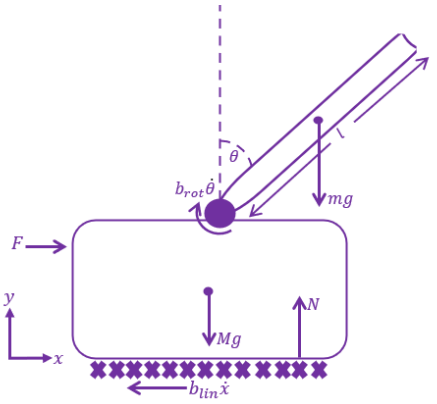

Figure 2.2: Free body diagram of the inverted pendulum system showing the applied force, gravitational forces, damping effects, and normal force acting on the system.

For the cart, the sum of forces in the horizontal direction (F_x) was established using Newton's second law:

$$\sum F_x = M\ddot{x} \tag{2.1}$$

For the pendulum, both translational and rotational dynamics were considered. The pendulum experiences gravitational forces, reaction forces from the cart, and inertial effects caused by the cart motion.

To eliminate the internal reaction forces between the cart and pendulum, the moment equation was derived around the pendulum center of mass. This resulted in a coupled set of nonlinear differential equations describing the dynamic behavior of the system.

The obtained equations contain nonlinear trigonometric terms originating from the pendulum rotation. To simplify the controller design, the equations were linearized around the upright equilibrium position using the small-angle approximations:

$$\sin(\theta) \approx \theta \tag{2.2}$$

$$\cos(\theta) \approx 1 \tag{2.3}$$

After linearization, the system equations were transformed into transfer functions and state-space representations suitable for simulation and controller design in MATLAB/Python.

Final Linearized Model

$$(I+ml^2)\ddot{\theta} - mgl\theta = ml\ddot{x} \tag{2.4}$$

$$(M+m)\ddot{x} + b\dot{x} - ml\ddot{\theta} = u \tag{2.5}$$

State-Space Representation

$$\dot{x} = Ax + Bu \tag{2.6}$$

$$y = Cx + Du \tag{2.7}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-b(I+ml^2)}{(M+m)I+Mml^2} & \frac{m^2gl^2}{(M+m)I+Mml^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-mlb}{(M+m)I+Mml^2} & \frac{mgl(M+m)}{(M+m)I+Mml^2} & 0 \end{bmatrix} \tag{2.8}$$

$$B = \begin{bmatrix} 0 \\ \frac{I+ml^2}{(M+m)I+Mml^2} \\ 0 \\ \frac{ml}{(M+m)I+Mml^2} \end{bmatrix} \tag{2.9}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2.10}$$

$$D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{2.11}$$

2.1.2 Euler-Lagrange Modelling

The Euler-Lagrange method describes the system dynamics using energy relationships instead of direct force balances. The method is based on the kinetic and potential energy of the system. [6]

The kinetic energy of the cart (T_{cart}) is given:

$$T_{cart} = \frac{1}{2}M\dot{x}^2 \tag{2.12}$$

The pendulum contributes both translational and rotational kinetic energy due to its motion relative to the cart. Additionally, the pendulum possesses potential energy caused by gravity.

Using the derived expressions for kinetic (T) and potential energy (V), the Lagrangian (L) was formulated as:

$$L = T - V \tag{2.13}$$

The equations of motion were then derived using the Euler-Lagrange equation:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = Q_i \tag{2.14}$$

where q_i represents the generalized coordinates of the system.

By applying the Euler-Lagrange formulation to both the cart position and pendulum angle, a coupled nonlinear dynamic model of the system was obtained.

Similar to the Newton-Euler approach, the nonlinear equations were linearized around the upright equilibrium position to simplify further analysis and controller design.

Kinetic and Potential Energy

$$E_{kin} = E_{kin,c} + E_{kin,p} + E_{kin,rot} \tag{2.15}$$

$$= \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m\dot{x}^2 - ml\dot{\theta}\dot{x}\cos\theta + \frac{1}{2}ml^2\dot{\theta}^2 + \frac{1}{2}I\dot{\theta}^2 \tag{2.16}$$

$$E_{pot} = mgl\cos\theta \tag{2.17}$$

Nonlinear Equations of Motion

$$-ml\ddot{x}\cos\theta + ml^2\ddot{\theta} + I\ddot{\theta} - mlg\sin\theta = -b_{rot}\dot{\theta} \tag{2.18}$$

$$(M+m)\ddot{x} - ml\ddot{\theta}\cos\theta + ml\dot{\theta}^2\sin\theta = F - b_{lin}\dot{x} \tag{2.19}$$

Linearized Equations

$$ml^2\ddot{\theta} + I\ddot{\theta} - mlg\theta + b_{rot}\dot{\theta} = ml\ddot{x} \tag{2.20}$$

$$(M+m)\ddot{x} - ml\ddot{\theta} + b_{lin}\dot{x} = u \tag{2.21}$$

Transfer Function

$$\frac{\Theta(s)}{U(s)} = \frac{1}{\left((M+m)\cdot\frac{ml^2s^2+Is^2-mlg+b_{rot}s}{mls^2}\right)s^2 - mls^2 + b_{lin}\left(\frac{ml^2s^2+Is^2-mlg+b_{rot}s}{mls^2}\right)s} \tag{2.22}$$

2.1.3 Comparison of the Methods

Both modelling approaches resulted in equivalent dynamic models after linearization, verifying the correctness of the derivations.

The Newton-Euler method provides a more intuitive understanding of the physical forces and moments acting on the system, since it directly uses force and torque balances. However, the method becomes increasingly complex for systems with multiple interconnected bodies.

The Euler-Lagrange method provides a more systematic and compact mathematical formulation by describing the system through energy relationships. This makes the method particularly useful for more advanced multibody systems.

The final linearized model obtained from the derivations was used as the foundation for the controller design and simulations presented later in this project.

The final linearized model was implemented in MATLAB/Python and used for simulation and controller development. Both transfer function and state-space representations were used throughout the project for analysis and validation of the implemented controllers.

The derived model forms the basis for the PID controllers, pole-placement controller, and the discrete implementation used later in the project.

3 Control Theory

There are many controllers and strategies that can be used to control and stabilize various systems. For an inverted pendulum attached to a cart on a belt, it is important to regulate both the pendulum and cart position since it is preferred that the cart stands still while the pendulum is balanced in its upward position and because the belt is limited in length.

3.1 LQR Controller

One way to stabilize the inverted pendulum in its upward position, is a Linear Quadratic Regulator (LQR). LQR is an "optimal" multivariable controller that determines the best possible feedback gains by balancing the trade off between keeping the system states close to zero and minimizing the control effort used to do so.

3.1.1 Discretization for PLC

Because the controller will be running on a PLC running at a fixed cycle time of $T_s = 0.001$ seconds, the continuous time statespace model cannot be used directly.
The system matrices (**A**, **B**, **C**, and **D** from section 2.1.1) are first discretized using a Zero-Order Hold (ZOH) method. This converts the physical model into a discrete-time representation that aligns with the sampling of the PLC.

3.1.2 The Cost Function and Weighting Matrices

The discrete LQR algorithm computes an optimal gain vector (**K**) by minimizing the discrete quadratic cost function (J):

$$J = \sum_{k=0}^{\infty} \left(\mathbf{X}_k^T \mathbf{Q} \mathbf{X}_k + u_k^T \mathbf{R} u_k \right) \tag{3.1}$$

Where:

- $\mathbf{X}_k$ is the state vector at time step k, defined as $\mathbf{X} = [x, \dot{x}, \theta, \dot{\theta}]^T$.
- u_k is the control effort (motor force).
- **Q** is a matrix that penalizes deviations in the system states (position and velocity errors).
- **R** is a matrix that penalizes the use of the actuator (control effort).

3.1.3 Tuning with Bryson's Rule

Bryson's Rule is applied here to decide what to put in the **Q** and **R** matrices in an intuitive way. This method scales each parameter based on its maximum acceptable physical limit, ensuring every variable contributes evenly to the cost function.

The maximum limits defined for this system are:

- **Cart Position (x):** 0.5 m

- **Cart Velocity ($\dot{x}$):** 1.0 m/s
- **Pendulum Angle (θ):** $\frac{\pi}{4}$ rad
- **Pendulum Velocity ($\dot{\theta}$):** 5.0 rad/s
- **Output Force (u):** 14.4 N

Using these limits, the diagonal values of the **Q** and **R** matrices are calculated as the inverse square of the maximum acceptable values ($\frac{1}{\text{max}^2}$).

3.1.4 The Control Law

Solving the algebraic Riccati equation with the weight matrices (**Q** and **R**) and discrete system matrices (**A** and **B**), gives the optimal feedback gain vector (**K**). In the PLC implementation, the control force applied to the cart is calculated simply by multiplying the static gain vector (**K**) by the current sensor states (**X**):

$$u = -\mathbf{KX} = -(K_1 x + K_2 \dot{x} + K_3 \theta + K_4 \dot{\theta}) \tag{3.2}$$

3.1.5 Poles & Performance for LQR

To evaluate the theoretical performance of the LQR controller, the closed-loop poles are analyzed. The calculated poles are:

$$p_1 = -44.45, \quad p_{2,3} = -5.98 \pm 0.68i, \quad p_4 = -1.95 \tag{3.3}$$

Because all poles have negative real parts, the system is considered stable. To get a measurement of performance, the properties of select poles are calculated. The real pole closest to the imaginary axis (p_4) dictates the slower settling behavior of the cart, while the complex pole pair determines the faster dynamics of the pendulum balancing. The last pole (p_1) is much further from the imaginary axis, which means it has very little impact on the performance of the system.

First, the complex pole pair ($s = -\sigma \pm j\omega_d$) is analyzed to determine the damping ratio (ζ) and natural frequency (ω_n):

$$\omega_n = \sqrt{\sigma^2 + \omega_d^2} = \sqrt{5.98^2 + 0.68^2} \approx 6.02 \text{ rad/s} \tag{3.4}$$

$$\zeta = \frac{\sigma}{\omega_n} = \frac{5.98}{6.02} \approx 0.993 \tag{3.5}$$

Because $\zeta \approx 1$, the inner pendulum is nearly critically damped. Which means there is close to no overshoot ($M_p \approx 0$).

Next, the Rise Time (t_r, 10% to 90%) is estimated using a second-order approximation on the pole pair representing the pendulum's swinging:

$$t_r \approx \frac{1.8}{\omega_n} = \frac{1.8}{6.02} \approx 0.30 \text{ seconds} \tag{3.6}$$

Lastly, since the pendulum being upright is dependant on the cart settling. The Settling Time (t_s, $\alpha = 2\%$) is estimated with another second-order approximation, this time on the dominant pole (p_4) representing the cart settling.

$$t_s \approx \frac{-\ln(\alpha)}{\sigma_{p_4}} = \frac{3.912}{1.95} \approx 2.01 \text{ seconds} \tag{3.7}$$

3.2 LQI Controller

While standard LQR is very effective at stabilizing the system, it functions purely as a proportional controller. This means it can suffer from steady-state errors if there are unmodelled dynamics or constant disturbances in the system. This could be friction in the belt which the cart sits on, inclinations in the setup or other forces affecting the cart or pendulum.

3.2.1 Augmenting the State Vector

To eliminate these steady state offsets, the controller can be expanded into a Linear Quadratic Integral (LQI) controller. This is achieved by introducing a new state variable that accumulates the cart position error over time, functioning like the integral term in a PID controller.

This new integral state is defined as:

$$x_i = \int_0^\tau (x_{ref} - x)dt \tag{3.8}$$

Where τ is the current moment in time. The standard state vector is then augmented from four variables to five:

$$\mathbf{X} = \left[x, \dot{x}, \theta, \dot{\theta}, x_i\right]^T \tag{3.9}$$

3.2.2 The Augmented Control Law

To implement LQI, the discrete system matrices and weight matrices are expanded to fit this fifth state. The augmented $\mathbf{Q}$ matrix receives a new diagonal value specifically to penalize the integral error. The maximum of this value is set to 1.0 and included in the new $\mathbf{Q}$ using Bryson's rule.

Solving the algebraic Riccati equation again for this augmented system gives the new gain vector, and the resulting control law becomes:

$$u = -\mathbf{KX} = -(K_1 x + K_2 \dot{x} + K_3 \theta + K_4 \dot{\theta} + K_5 x_i) \tag{3.10}$$

Because the $K_5 x_i$ term continuously builds up control effort as long as a there is an error in the cart position, it ensures that the cart will reach its target position, regardless of small physical imperfections, friction or other forces.

3.2.3 Poles & Performance for LQI

Adding the integral state introduces a fifth pole to the closed-loop system. The calculated poles for the augmented controller is:

$$p_1 = -43.97, \quad p_2 = -6.37, \quad p_3 = -5.73, \quad p_4 = -1.90, \quad p_5 = -0.52 \tag{3.11}$$

Like the LQR, all poles have negative real parts, meaning the system is stable. Also, all five poles have no imaginary part, this indicates that the theoretical response can only be overdamped, ensuring no overshoot ($M_p = 0$). In the LQR controller, the pendulum movements were decided by the complex pole pair at $-5.98 \pm 0.68i$, and the Rise Time (t_r) was found from the natural frequency (ω_n) this results in. Instead, in the LQI controller, this pair splits into two real poles without an imaginary part:

$$p_2 = -6.37, \quad p_3 = -5.73 \tag{3.12}$$

The speed of the pendulum balancing is limited by the slower of these two poles ($p_3 = -5.73$). By applying the rise time definition (the time to travel from 10% to 90% of the reference) to this specific local dominant pole ($\sigma_{p_3} = 5.73$), the pendulum's theoretical rise time can be estimated:

$$t_{0.1} = \frac{-\ln(0.90)}{\sigma_{p_3}} \approx 0.018 \text{ seconds} \tag{3.13}$$

$$t_{0.9} = \frac{-\ln(0.10)}{\sigma_{p_3}} \approx 0.402 \text{ seconds} \tag{3.14}$$

$$t_r = t_{0.9} - t_{0.1} \approx 0.384 \text{ seconds} \tag{3.15}$$

Next, to find the Settling Time, the speed at which the cart settles must be found. However, the new integral pole (p_5) is now the dominant pole of the system, but since the cart pole (p_4) is what affects the system for most of the time this will be what best estimates the Settling Time (t_s). Calculating it based on this gives:

$$t_s \approx \frac{-\log(0.02)}{\sigma_{p_4}} \approx 2.06 \text{ seconds} \tag{3.16}$$

3.3 Cascaded PID Controller

While modern statespace methods like LQR are very effective for multivariable systems, classical control techniques can also be applied. Since regulating both the cart position and the pendulum position is required to balance the pendulum on the limited belt length, a single controller will not be enough. Instead, two cascaded regulators is used, one being a Proportional-Derivative (PD) controller for the pendulum and the other, a Proportional-Integral-Derivative (PID) controller for the cart.

3.3.1 Cascade Setup

The control system is divided into two nested feedback loops:

- **Inner Loop (Pendulum):** This loop focuses entirely on stabilizing the pendulum angle (θ). It uses a PD controller. The integral term is not used here, as it can introduce phase lag and worsen the stability of the highly unstable system of the inverted pendulum. The derivative term provides the critical damping required to catch and balance the pendulum.
- **Outer Loop (Cart):** This loop controls the cart's position (x) using a full PID controller. Instead of outputting force directly, it calculates the necessary pendulum angle (θ_{ref}) required to move the cart. The integral term is used in this outer loop to eliminate steady state positional errors caused by track friction or unmodeled slight inclinations in the setup.

For this cascade to function, the inner angle loop must be tuned to react significantly faster than the outer position loop, preventing the two controllers from fighting each other.

3.3.2 Tuning via Root Locus

A method for tuning this type of setup is the Root Locus technique, which visualizes the paths of the closed-loop poles as a control gain is varied.

The transfer function for the inner PD controller is:

$$C_{PD}(s) = K_{p,\theta} + K_{d,\theta}s \tag{3.17}$$

This structure adds a single zero to the open loop system. The inner loop is tuned first by strategically placing this zero on the s-plane to "pull" the unstable open loop poles of the pendulum into the stable left-half plane (LHP), ensuring the system meets the performance requirements.

Once the inner loop is closed and verified to meet the goals, the outer loop is tuned. The transfer function for the outer PID controller is:

$$C_{PID}(s) = K_{p,x} + \frac{K_{i,x}}{s} + K_{d,x}s = \frac{K_{d,x}s^2 + K_{p,x}s + K_{i,x}}{s} \tag{3.18}$$

The PID controller introduces one pole at the origin (from the integrator) and two zeros. Using Root Locus on the new combined system, these zeros are placed to change the trajectory of the cart's poles. The goal here is to ensure the cart moves slowly so that the pendulum is not destabilzed by aggressive changes to the pendulum angle (θ_{ref}).

3.3.3 Cascade Control Law

First, the cart's position error (e_x) is calculated by subtracting its current measured position (x) from the target position (x_{ref}):

$$e_x = x_{ref} - x \tag{3.19}$$

The error is multiplied by the proportional gain ($K_{p,x}$), its accumulated sum over time is multiplied by the integral gain ($K_{i,x}$), and its rate of change is multiplied by the derivative gain ($K_{d,x}$). The sum of these three terms does not output a motor force. Instead, it gives a reference angle (θ_{ref}), which is the amount the pendulum needs to lean to move the cart toward its target position:

$$\theta_{ref} = K_{p,x}e_x + K_{i,x}\int_0^\tau e_x dt + K_{d,x}\frac{de_x}{dt} \tag{3.20}$$

Once the reference angle (θ_{ref}) is calculated by the outer loop, the inner loop calculates the pendulum's current angular error (e_θ):

$$e_\theta = \theta_{ref} - \theta \tag{3.21}$$

The angular error is then multiplied by the proportional gain ($K_{p,\theta}$), and its rate of change is multiplied by the derivative gain ($K_{d,\theta}$). Adding these two terms gives the final control law, which outputs the force (u):

$$u = K_{p,\theta}e_\theta + K_{d,\theta}\frac{de_\theta}{dt} \tag{3.22}$$

3.3.4 Poles & Performance for Cascade

Analyzing the closed-loop poles of the cascaded PID-PD structure shows significant theoretical stability issues. The calculated poles are:

$$p_1 = 1.0853, \quad p_2 = 0.0751, \quad p_3 = -0.1163, \quad p_4 = -0.0743 \tag{3.23}$$

$$p_5 = -0.0643, \quad p_{6,7} = 0.0031 \pm 0.0215i, \quad p_8 = -0.0089$$

Performance metrics such as Rise Time, Settling Time, and Overshoot are only defined for stable systems. Because this system possesses multiple poles with positive real parts (p_1, p_2, and the $p_{6,7}$ pole pair), the model of the system is considered unstable. However, it must be stated that the controller does not exhibit this behaviour on the actual setup.

3.4 Swing up

Something that is not necessary for balancing the pendulum itself is swing up, instead the purpose of this is to get the pendulum from hanging downward to within a specific range of the upward position where a controller can catch it. This makes the testing process of the different controllers much easier and the overall implementation and use of the system simpler.

3.4.1 The Energy Controller

To achieve the swing up, the system uses an energy based controller. This injects kinetic energy into the pendulum until its total mechanical energy equals the potential energy required to stand upright.

The total mechanical energy (E) of the pendulum is calculated as the sum of its kinetic and potential energy:

$$E = \frac{1}{2} I_p \dot{\theta}^2 + m_p g l (\cos\theta - 1) \tag{3.24}$$

The system defines the upward position as having an energy state of 0.0.

By subtracting 1.0 from the cosine of theta, the potential energy is offset so that it peaks at zero when the pendulum is in the upward position. The controller then calculates the energy error (e_E) as the difference between this target state (0.0) and the pendulum's current mechanical energy:

$$e_E = 0.0 - E \tag{3.25}$$

To put more energy into the pendulum, the controller must accelerate the cart in step with the pendulum's swing. The direction (s) of this acceleration is determined by the sign of the product of the angular velocity and the cosine of theta:

$$s = sign(\dot{\theta} \cos\theta) \tag{3.26}$$

3.4.2 The Control Law & Exit Condition

The control effort (u) is then calculated using the energy error (e_E), an energy gain constant (k_E), and the direction (s). However, just putting in energy to the pendulum could cause the cart to drift so a small spring force is added to gently center the cart ($-kx$):

$$u = s k_E e_E - kx \tag{3.27}$$

The swing up PLC state always monitors the pendulum's position and velocity to figure out if it is safe to hand over to a controller. This "catch condition" triggers only when two conditions are met at the same time:

- The position is within $\pm 30^\circ$ of the upward position ($|\theta| < \frac{\pi}{6}$ rad).
- The velocity is low enough for a controller to catch ($|\dot{\theta}| < 1$ rad/s).

Once the conditions are met all errors and variables from controllers are reset and the PLC state is switched to controller to balance the pendulum.

4 PLC Implementation

Moving from a simulated environment to a physical one comes with a list of challenges that need to solved. Firstly, control of the cart must be understood. The cart is driven along a rail by a belt, attached to an AC servomotor. The motor is connected to a servo drive which in turn is connected to the PLC via an analog signal. The servo drive can be set to a target torque mode, which aims to hit a percentage of the maximum torque of the motor based on the analog input. With this information, a scaling factor can be used when the PLC needs to calculate how much voltage needs to be send via the analog signal to apply a certain torque in newton on to the cart. Based on the belt's roller diameter and the motor's max torque, the maximum force can be found:

$$force_{max} = \frac{torque_{max}}{roller_{diameter}} = \frac{0.72Nm}{0.05m} = 14.4N \tag{4.1}$$

With the voltage input of the servo drive being ± 10 V, a simple conversion rate from controller output (u) to the required voltage output can then be found:

$$voltage_{required} = \frac{force_{max}}{voltage_{max}} = \frac{14.4N}{10V} = 1.44\frac{N}{V} \tag{4.2}$$

In order to interact with the controllers, the inputs from the encoders must be converted. The encoder for the pendulum has a resolution of 4096 units per full cycle and is reset such that the encoder shows 0 when the pendulum is hanging straight down. The controller expects the input to be in radians with 0 pointing straight up. Thus the conversion to θ can be done as follows:

$$\theta = ((\theta_{encoder} \bmod 4096) - 2048) \cdot \left(\frac{\pi \cdot 2}{4096}\right)[rad] \tag{4.3}$$

The conversion for the cart position x was found using a measured distance. Moving the cart from one end of the track to the other resulted in the encoder showing 381,709 units while the cart moved 156.7 centimeters. Thus the amount of units per meter can be found as:

$$\frac{381,709units}{1.567m} = 243,592\frac{units}{m} \tag{4.4}$$

The controllers expect x to be in meters, with 0 at the center of the track, which is set to 191,000 units, roughly half of the full length. The conversion looks as follows:

$$x = \frac{x_{encoder} - 191,000}{243,592}[m] \tag{4.5}$$

The controllers also need to be provided with the cart velocity $\dot{x}$ and angle velocity $\dot{\theta}$. Since the encoder data is the only measurement provided from the physical system, these values have to be calculated. The simplest way to calculate velocity at the current time step k is to look at the change in position between the current and most recent step. However, with a cycle time T_s of 1 ms, even small changes and inconsistencies can quickly have a large impact on the velocity when comparing two consecutive time steps. Therefore, a filter

with a scaling factor α is implemented as part of the velocity calculations, which takes the following form:

$$\dot{\theta}_k = \frac{\theta_k - \theta_{k-1}}{T_s} \tag{4.6}$$

$$\dot{\theta}_{k,filtered} = (\alpha \cdot \dot{\theta}_k) + (1.0 - \alpha) \cdot \dot{\theta}_{k-1,filtered} \tag{4.7}$$

The velocity for the cart is calculated using the same formula with x instead of θ.

4.1 LQR

Having calculated x, $\dot{x}$, θ and $\dot{\theta}$, implementing the LQR controller is somewhat simple. In each cycle the output force u is found by using the equation 3.2. This provides a force in newton which can be converted to a voltage output using equation 4.2. The voltage is sent to the servo drive each cycle, affecting the cart with the desired force.

4.2 LQI

LQI has a rather similar implementation to LQR, using equation 3.10. The major difference being the addition of an integral term as seen in equation 3.8. The integral is calculated using backward integration in each cycle as follows:

$$x_{i,k} = x_{i,k-1} + x_k \cdot T_s \tag{4.8}$$

As x describes the current distance from the reference 0.0, it essentially acts as the error of the cart position e_x. One possible issue with integral terms is their ability to grow endlessly if an error persists. This would push the controller to output increasingly higher target forces, which may not be desirable. In order to limit this effect, a simple anti wind-up mechanism is implemented. In practice $x_{i,k}$ is checked each cycle and clamped to a max value if it exceeds it.

4.3 PID

The cascaded PID controller has a slightly more involved implementation. Two similar control loops are set up to calculate the inner and outer loop respectively. Firstly, the outer cart loop needs to run to find the reference point for the inner pendulum loop. As shown in equation 3.20, 3 terms are necessary for this calculation, namely e_x, $\int_0^\tau e_x dt$ and $\frac{de_x}{dt}$. As mentioned, e_x is the same value as x in the PLC implementation, thus the first term is satisfied. The integral term, can be satisfied by using equation 4.8. That leaves the derivative term, which is found using equation 4.7 with x swapped in θ's place. The implemented calculation looks as follows:

$$\theta_{ref,k} = K_{p,x} \cdot x_k + K_{i,x} \cdot x_{i,k} + K_{d,x} \cdot \dot{x}_{k,filtered} \tag{4.9}$$

The inner loop is calculated in a similar fashion, with two notable differences. As the controller does not have an integral gain this part of the calculation is left out. As reference for θ is not a static 0.0, the θ equation 4.3 cannot be used in place of e_θ, instead equation 3.21 is used. The final calculation for each cycle becomes:

$$u_k = K_{p,\theta} \cdot e_{\theta,k} + K_{d,\theta} \cdot \dot{\theta}_{k,filtered} \tag{4.10}$$

4.4 Safety

To prevent mechanical damage and ensure safe operation, two layers of protection are implemented. First, actuator saturation is applied, limiting the control effort to 8.8 N. This restricts the cart's maximum acceleration, preventing it from striking the track's end-stops at excessive speeds. Second, a software safety monitor acts as an emergency watchdog; if the state variables (position or angle) exceed predefined safety thresholds, the system immediately disables the servo drive.

5 Data Collection

To gather data for actual comparison a system was built utilizing OPC UA, Python and Matlab Simulink. OPC UA takes the data directly from the PLC program, sends it to the Python script, which interprets and stores the data in a CSV file.

5.1 OPC UA

OPC Unified Architecture (UA) is an industrial, cross platform communication protocol. It is widely used to control and communicate with devices such as PLCs, servers, micro controllers and more. In this project it was used because it is built in to Automation Studio and the B&R PLC used. This allowed a direct connection between the PLC and a PC used to collect the data. Because Eduroam is such a restricted network, and the only proper wifi available at the university, a physical Ethernet connection was necessary. Though not necessary in this case, without this limitation, it would be possible to send the data wirelessly, and potentially from across the world. With this connection it was possible to transfer the values of any variable needed. The variables transferred were the state of the program, the step the data was captured, the cart position, the angle of the pendulum, and the force applied to the motor. According to the documentation from B&R, it is not possible to send data faster than 100hz [8], but in practice this was not true, more about this can be found in the discussion 7.3

5.2 Python Script

To interpret and save the data, a small Python script was made. It takes the IP from the PLC and connects via OPC UA. The data is picked up in a datastream using the asyncua library [7], and then stored in the memory of the receiving PC. When the entire test is done, defined by a time limit, all the data is written from memory to a Comma Seperated Values file (CSV).

5.3 Simulation

To simulate the controllers, MATLAB and Simulink are used. The MATLAB script is a copy of the PLC implementation of the PID cascade, where the simulation force is inputted into a state-space model instead of the physical setup. It is done this way because, in all attempts to use Simulink, the force went toward infinity. LQR was implemented in Simulink by using a state-space block, where the input is force u, and the output is the state, which is used to find u, see section 3.2. The LQI implementation in Simulink is an extension of LQR, where x is sent through an integrator block and the gains are changed.

5.4 CSV and Excel

To read all the data, Microsoft Excel can import all the data written to the CSV files. For easier readability and data processing; the entire Excel file is attached with the code in appendix B. All the test data captured is stored, with more graphs than included in this report.

6 Evaluation

To evaluate the different controllers, multiple tests showing off their individual strengths and weaknesses were done. All of the controllers were tested multiple times to ensure a good sample size, the data was treated and cleaned, and then a set of data deemed to be the most representative was chosen for each controller and for each test. All the data can be found in appendix B, but graphs of the representative data can be seen here.

For each test, 3 graphs were made. The cart position shows how the cart reacts and moves during the various stages of tests. For the cart position graphs, the calculated x position in meters is used, with 0 at the center of the track. The pendulum angle shows what the angle of the pendulum is during the tests. This is shown in degrees measured by the encoder sitting at the base of the pendulum in the cart's center. The motor force parameter is a readout of the force applied by the motor. It is measured in newtons, as the motor drive has a mode specifically for torque, as mentioned in chapter 4.

All tests are colour-coded, such that the controllers use separate colors, with the physical data shown in a full line, while the simulation uses a dotted.

6.1 Impulse Test

The impulse test was done by applying a single pulse of 6 newtons, for 50 milliseconds, to the output of the controller. This means that at an output of 1 newton, the cart would be affected by 7 newtons for 50 milliseconds. It is primarily used to show how the various controllers recover from one off pushes.

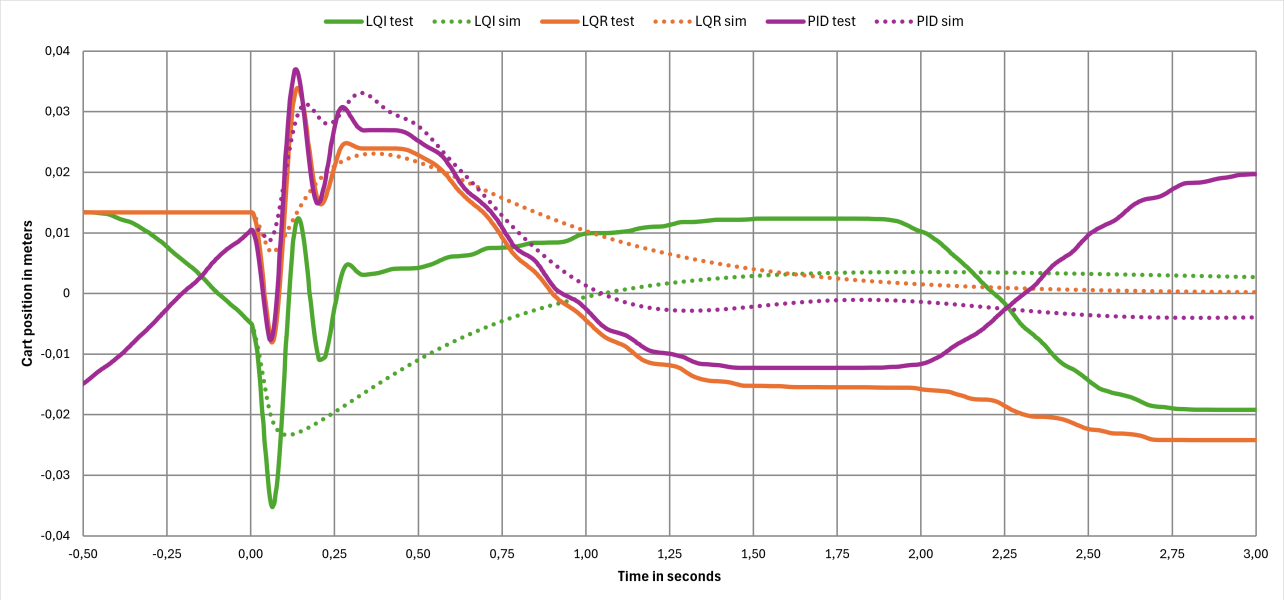

Figure 6.1: Cart position during the impulse test.

The impulse affects the 3 controllers differently, likely because of the difference in system state when the impulse happens. As seen in the graph, all 3 controllers are in a different starting state. The LQR is very steady, while the PID and LQI both oscillate a little. Even though they all move to a different spot, they still all move in a very similar way, with a sharp impulse that evens out to find a steady state again.

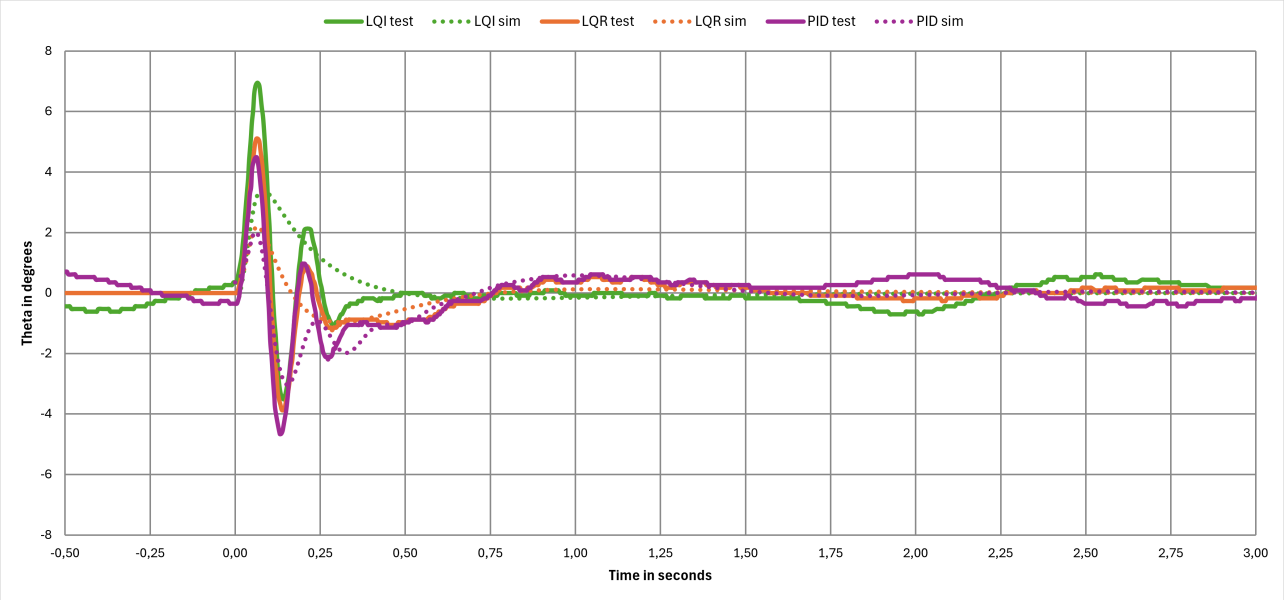

Figure 6.2: Pendulum angle during the impulse test.

The 3 controllers start by balancing the pendulum very close to 0, the fully upright position. When the impulse hits, they all swing wide one way, before compensating the other way, creating a small oscillation before a steady balance is reached again.

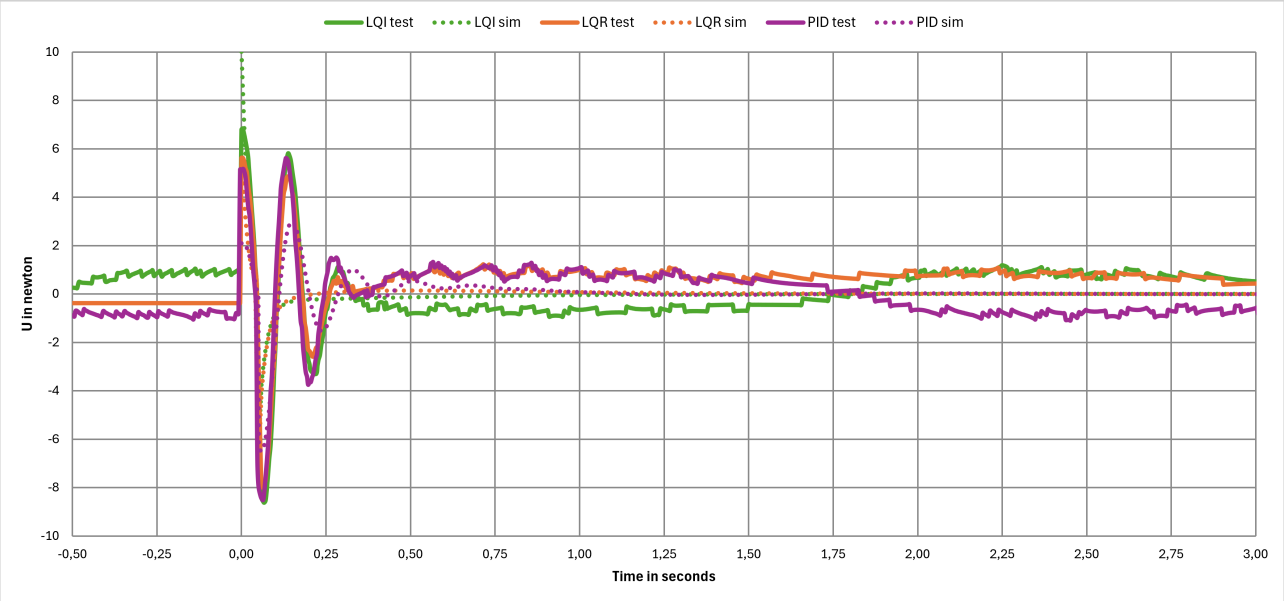

Figure 6.3: Motor force during the impulse test.

The motor force is very similar for all 3 controllers. It is very steady in the beginning, but with a slight offset. When the impulse hits, the force swings until it settles back down to a similar point as the starting level.

6.2 Step Response Test

The step test was done similarly to the impulse, but instead it was applied for 10 seconds rather than 50 milliseconds, and with 8 newtons of force. This allows certain controllers to show off their resilience under continued pressure.

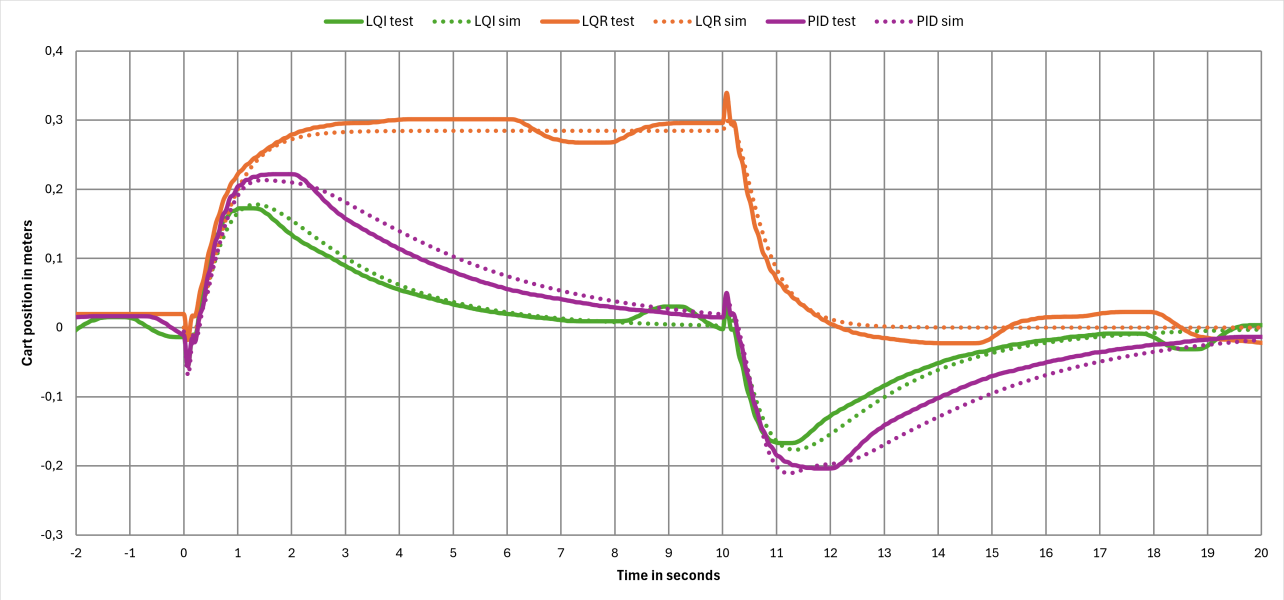

Figure 6.4: Cart position during the step response test.

The cart position in the step test shows the LQR controller moving to one side for the entire duration of the step. However both the LQI and PID controllers go to the side, then gravitate back towards the center, until the step is over, where it jerks to the opposite side, then settles back to the middle again.

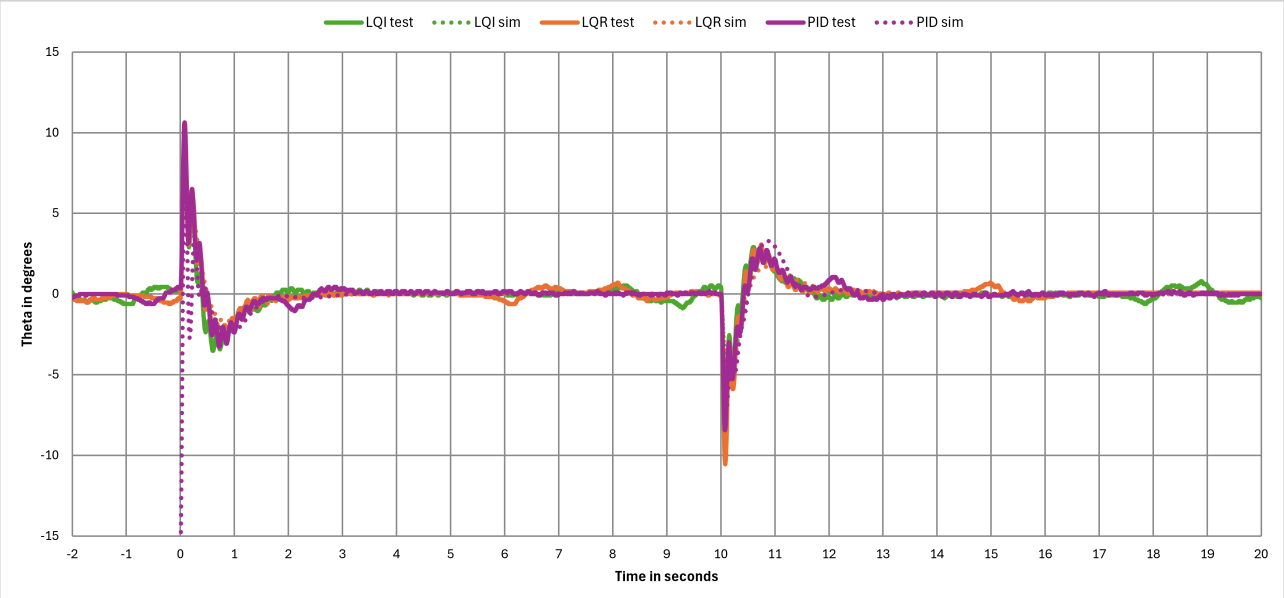

Figure 6.5: Pendulum angle during the step response test.

The pendulum has a very similar, yet opposite movement to the cart itself. At the start of the step, all 3 controllers swing to one side due to the push, and at the end of the step, they swing the oppositre way, as the step ends and the cart moves the other direction.

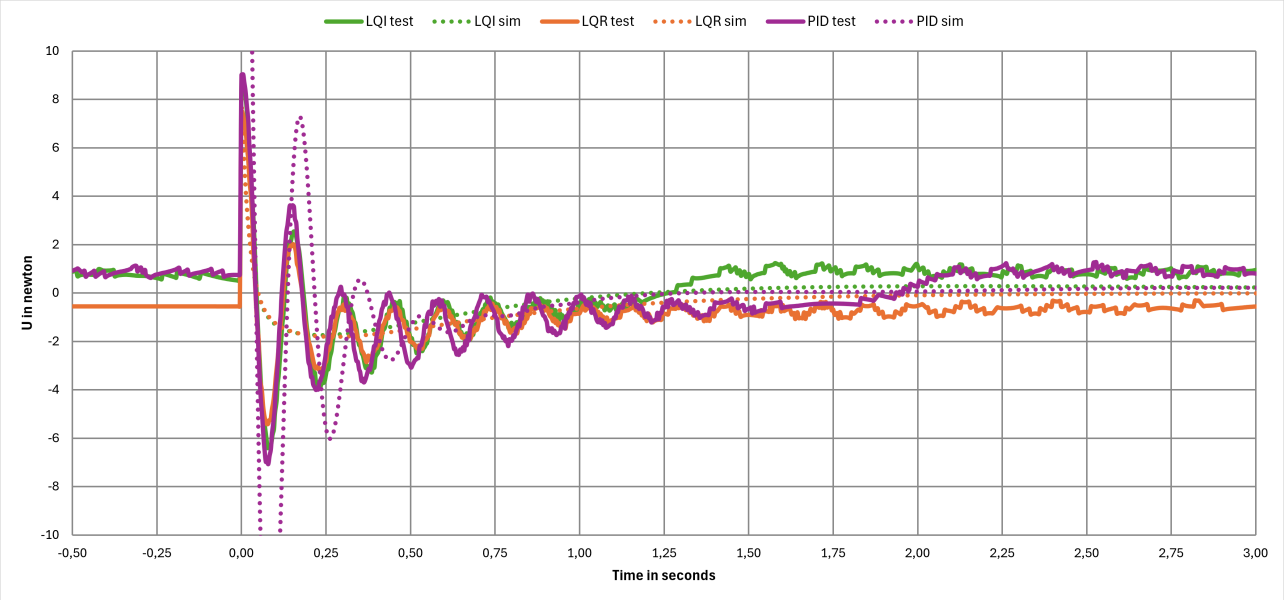

Figure 6.6: Motor force during the step response test.

The motor force during the step looks very similar for all 3 controllers. Both the LQI and PID have a different starting force, and go back to that same force after the step. Note that this graph only shows 3.5 seconds, but look similar at the end of the step. This can be seen in the data in appendix B.

6.3 Swing Up Test

Finally the swing up test, which was done by initiating the swing up, and switching to the controller when possible. This shows how quickly and well the controller can compensate, stabilizing an unstable pendulum. This is essentially an initial condition test, with a moving pendulum. One thing to note is that the graphs only show the time after the controller kicked in, and not during the swing up itself.

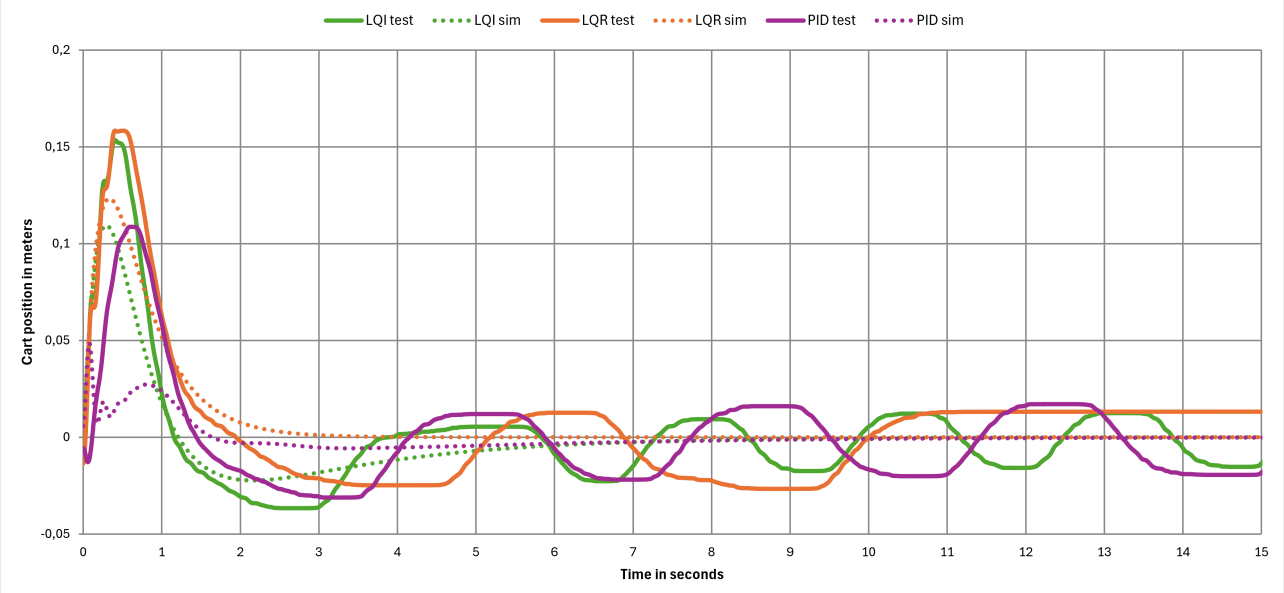

Figure 6.7: Cart position during the swing up test.

Because the controller kicks in just at the top of a swing with the angle being substantially off from the reference, all 3 controllers react similarly, with a larger movement to recover before settling.

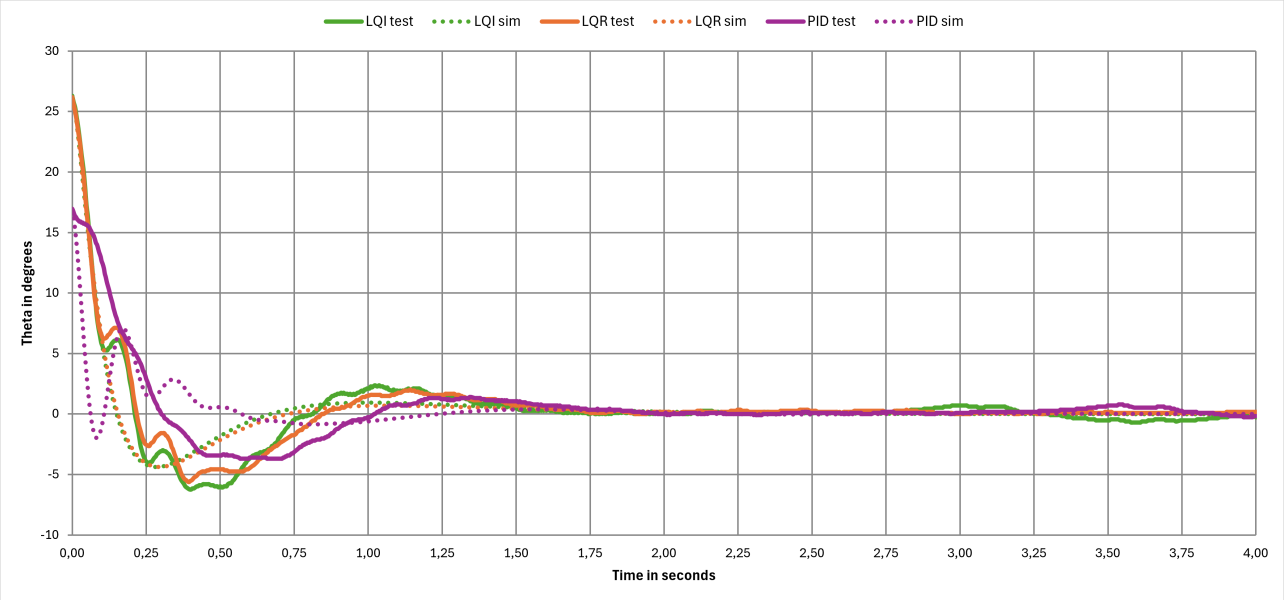

Figure 6.8: Pendulum angle during the swing up test.

The angle of the swing up reacts in the opposite direction of the cart position, as the cart is moved quickly to get ahead of the pendulum. This is shown in the graph as a sharp decline in the angle, before it evens out near the correct angle. Note that the graphs time is much shorter, with more detail, as to show the intricacies of the angle, that cannot easily be seen by looking at a longer period of time.

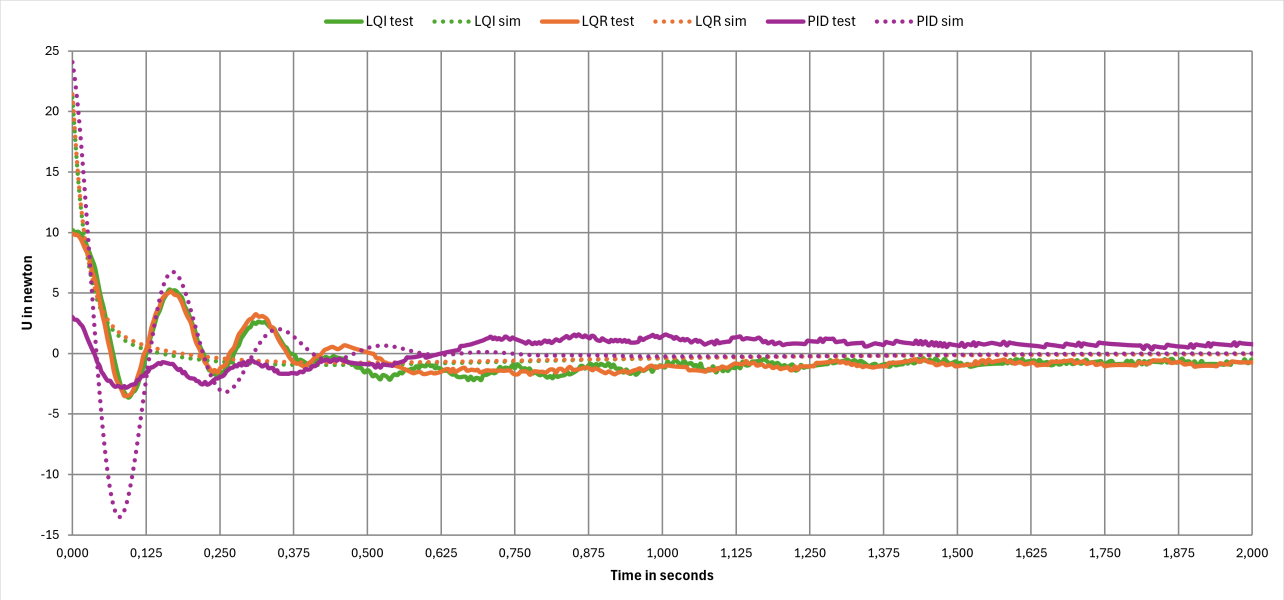

Figure 6.9: Motor force during the swing up test.

The motor force outputted during the swing up, like with the impulse, looks a bit different depending on how far up the swing got before the controller took over, and at what velocity. In practice this shows up as a different degree of spike during the switch, as the motor had to work less to catch the pendulum.

6.4 Performance Metrics

The performance metrics of the controllers on the physical system can be estimated from the test data. For this, the selected test sets used in the graphs will be analyzed.

6.4.1 LQI

The LQI controllers initial angle is 0.4663 rad which forms the baseline for the performance metric calculations. To estimate the Settling Time (t_s) the last time step where the angle is outside of 2% of the initial angle ($0.4663 \cdot 0.02 = 0.0093$ rad) must be found. This happens 1.49 seconds after the initial position. Thus:

$$t_s \approx 1.49s \tag{6.1}$$

The Rise Time (t_r) is found by identifying the time steps where the pendulum reaches 90% (0.4197 rad) and 10% (0.0466 rad) of the initial position, this happens after 0.02 and 0.2 seconds respectively. Thus:

$$t_r \approx 0.2 - 0.02 \approx 0.18s \tag{6.2}$$

The Overshoot (M_p) is found by locating the time step with the highest value with opposite sign to the initial value. This is found to be -0.1058 rad. Thus:

$$M_p \approx \frac{0.1058}{0.4663} \approx 22.7\% \tag{6.3}$$

6.4.2 LQR

The initial angle for the LQR controller is 0.4648 rad. The Settling Time at 2% (0.9296 rad) is found 1.6 seconds after the initial position. The Rise Time thresholds of 90% (0.4183 rad) and 10% (0.0465 rad) are found at 0.02 and 0.2 seconds respectively. The maximum overshoot is found to be 0.0966 rad. Thus:

$$t_s \approx 1.6s \tag{6.4}$$

$$t_r \approx 0.2 - 0.02 \approx 0.18s \tag{6.5}$$

$$M_p \approx \frac{0.0966}{0.4648} \approx 20.8\% \tag{6.6}$$

6.4.3 PID

The initial angle of the Cascaded PID controller is 0.3022 rad. The 2% Settling Time is found after 0.73 seconds at an angle of 0.006 rad. The Rise Time thresholds of 0.272 and 0.0302 rad are found at 0.05 and 0.27 seconds respectively. The maximum overshoot is found after 0.57 seconds at 0.0644 rad. Thus:

$$t_s \approx 0.73s \tag{6.7}$$

$$t_r \approx 0.27 - 0.05 \approx 0.22s \tag{6.8}$$

$$M_p \approx \frac{0.0644}{0.3022} \approx 21.3\% \tag{6.9}$$

7 Discussion

7.1 Implementation

The implementation of the controllers on the physical system does come with some challenges and limitations. One of the early important factors to consider was the minimum cycle time of the PLC. The default cycle time is set at 10 ms, which quickly showed itself as an issue. With the first attempt at getting a controller to work, a preliminary PID controller was tested at the default cycle time. It did not stabilized. After some trial and error, changing parameters, gains and the like, to no avail, it was found that the cycle time was the crucial issue. Lowering the cycle time to 2 ms yielded marginal stability, and pushing the hardware to its absolute limit of 1 ms successfully brought the loop latency down to an acceptable level.

While it was not achievable to get the pendulum stable at a 100 Hz sampling frequency during preliminary testing, it should theoretically be possible. One benefit to using a slower sampling rate is the reduction of noise impact. With a very fast sampling rate, any slight changes and inconsistencies in the data will impact the calculations substantially. A slower rate will act as a natural filter, smoothing some of the noise, but also increasing the delay of information.

The introduction of a filter on the velocity was found to be necessary during testing. The implementation uses an exponential moving average which can be tuned using a scalar α. It is important to note that this is trade-off between noise suppression and phase delay. It was vital in order to limit jittering, while the phase delay did not seem to impact performance to a point of instability even with an $\alpha = 0.01$. A simple equation shows why this filter is a necessity. As the encoder has 4096 units per cycle, if the encoder ticks by just one unit between cycles at 1 ms, the velocity change is:

$$\dot{\theta}_{min} = \frac{2\pi}{4096 \cdot 0.001} = 1.53 rad/s \tag{7.1}$$

The filter helps to smooth these significant velocity spikes by adding only a fraction of the velocity to the running average.

While the actuator saturation is a simple measure to prevent physical damage, it also impacts the controllers. In theory, the controllers assume an infinite linear actuator, but in reality, they rarely push the control effort beyond the set thresholds. However, as seen in the test data (see figure 6.9), there are occasions where the controller will output a control effort above 8.8 N. At these times, this safety measure is impacting the controller performance.

7.2 Model Mismatch

There are many reasons as to why the real data from the setups are not lining up with what the simulation's prediction. However, usually the main reason for this is a disconnect between the "physical statespace", that being the actual physical setup, motors and sensors, and the modelled statespace. For many reasons alot of details in the model are omitted as to simplify the calculations and intuition in the inner workings of the physics, as usually just modelling the main factors and physics is enough to create a working functioning implementation.

7.2.1 Unmodelled Friction

The system models rely on simplified linear damping coefficients. While this makes the math more manageable, it does not fully reflect the real world. In reality, the belt-driven conveyor system is experiencing static friction and slight belt backlash. The friction on the belt means there is deadband around the center point, where the controllers' output is too low to overcome the friction. This is especially notable in figure 6.7 around the LQR controller, which manages to stabilize the pendulum in an upright position to the degree that the system can be shut off without the pendulum falling over. However, this stable state is not at the true center, it is slightly off, and the control effort is not 0. In theory, the integral terms of the PID and LQI controllers should help compensate for this, however, this was not the case in reality. The cart position settles as the control effort goes to zero, but the two controllers with integral terms insist on trying to force the cart towards the true 0. As the static friction is not overcome instantly, the integral term continues to build until the friction is overcome, resulting in a quick acceleration of the cart. The cart overshoots the target reference before slowing down and ends up in a similar position on the opposite side of the reference, endlessly moving back and forth. Whereas, theoretically, the LQR controller should be the one experiencing slight oscillations while the LQI controller should be completely still according to the closed-loop pole's performance assesment found in section 3.2.3.

7.2.2 Energy Equations Used for Euler-Lagrange

Another significant source of model mismatch comes from the simplifications made during the Euler-Lagrange derivation in Python, this has been corrected in the report but it must be stated that the code, which found the various gains, with the oversimplification has been used for all the simulations and tests due to a lack of time to correct it. When calculating the kinetic and potential energy of the system, the pendulum was modelled as a point mass. This approach assumes that the entirety of the pendulum's mass is concentrated at its center of mass, which completely omits the rotational moment of inertia of the uniform metal rod spinning around its pivot. Furthermore, the physical setup includes a small bolt located at the tip of the pole. While the actual mass of this bolt is small and might seem negligible, its dynamic impact may not be. Because rotational inertia scales with the square of the distance from the pivot (mr^2), adding even a few grams at the furthest point of the pendulum could greatly increases the total moment of inertia and slightly shift the true center of mass upwards. However, usually most controllers are robust and stable enough to absorb the effects from this and still work.

7.3 Data Collection

One of the major hardware limitations of the PLC is its storage capacity. This means it is not viable to collect the data directly on the PLC to be moved later. Thus the data must be moved from the PLC to another device continuously, which is why an OPC UA system was implemented. The speed of the OPC UA communication is somewhat limited, and paired with python, slightly fluctuating. This shows itself in the test data as the data points are not collected at a fixed rate. Though the PLC sampled information every millisecond, the collected samples have 3-4 ms between them. In order to counteract this uncertainty, the samples are sent as the last thing during a cycle and a step count from the PLC is sent along with it. This means the state of system should match the step count and should be consistent in each data sample, allowing the data to be synchronized. While this solution is sub-optimal, it was determined to be acceptable for the testing required in this project.

7.3.1 Simulation

The biggest issue with the simulation is the PID cascade controller simulation. As it is a copy of the PLC controller, it never uses any transfer function. This gives reason to believe that the Simulink controller was unstable because the transfer function and PID-PD gains were not correct for simulation.

7.4 Test Results

The initial result that can be drawn from the tests, is that getting different types of controllers, using different strategies, to balance the inverted pendulum on the physical setup was achieved. Having said that, there are quite a few important nuances and details to be considered.

Firstly, it was not possible to get a working cascaded PID controller based on tuning via Root Locus alone. Using the gains found via Root Locus as a starting point, the controller was manually tweaked to be stable. This was done by adjusting the gains individually until the physical system would stabilize and the performance was somewhat comparable to that of the LQR and LQI controllers. Why using the gains from Root Locus directly was unsuccessful is still unclear. It is likely that some or all of the factors discussed previously will have impacted the controller to the extent of instability. To be clear, this means that all the test data for the cascaded PID controller was done using the manually tuned gains.

The data from the impulse tests show a rather consistent picture across the controllers, with all three acting similarly to the simulations. The primary thing to note here, is the increased oscillation seen across the board. This could likely be due to friction on the physical system, which as mentioned was only considered at a simplified level in the modeling. In pure mathematics, a true impulse is modeled as a Dirac delta function ($\delta(t)$), which has an infinite amplitude, a width of zero and a total area under the curve of exactly 1. Since infinite force and zero time are physically impossible, the short pulse of 6 newton was applied to the system, imitating an impulse. This could also be described as a Shock Disturbance test.

As the step test was executed using an applied force, it technically is not a true Step Response test. Rather it becomes a sustained force disturbance test. Unfortunately, this was discovered late in the project and re-running tests was not possible due to time constraints. That being said, the data from the performed test does show similar results to the impulse test. All three controllers exhibit behavior closely comparable to the simulation. Again, the main difference is the increasing oscillation of the physical controllers, due to the same likely reasons.

The swing up test was introduced as way to analyze initial state behavior. This gives an indication to the performance of each controller when applied to a system in an unbalanced state. As mentioned in section 7.2.1, this test makes it clear that friction has a huge impact on the system performance.

7.5 Performance Metrics

Looking at the performance calculations on the tests in section 6.4, it can be seen that the numbers somewhat closely matches the simulated performances (found in sections 3.1.5 and 3.2.3), at least in the fact that LQR and LQI have similar performance. However, using this swing up test is a flawed way to find the performance as the swing introduces velocities and other unconventional changes in the state that the performance metrics are not well defined for. A true step test could have been done by instead shifting the cart's reference point on the track while controllers were active. A benefit to this would have been cleaner initial positions to use in the estimation of performance metrics on the physical system. So, these measurements can only loosely be compared with simulations. This also results in an overshoot which is not seen in the theoretical performance.

Comparing the Settling Time (t_s) values found in simulation and measurement it is shown that they are about 0.5 seconds off from eachother, this does seem to match fine when taking the flawed test into account as they are of similiar orders of magnitude. The same can be seen with the Rise Time (t_r), here again they are off by about 0.1 seconds and are of the same order of magnitude. This is applicable to both LQR and LQI, however nothing can be said of the Cascade controller as the close-loop pole analysis was unsuccessful.

8 Conclusion

The purpose of this project was to model, simulate and control an inverted pendulum mounted on a conveyor-driven cart system. The project combined theoretical modelling, controller design, simulation, PLC implementation and experimental validation of the physical setup. Due to the unstable and nonlinear nature of the inverted pendulum system, the project provided both theoretical and practical challenges related to modern control engineering and automation systems. Mathematical models of the system were developed using both the Newton-Euler and Euler-Lagrange methods. Although the two approaches were derived from different physical principles, both methods resulted in equivalent dynamic models after linearization around the upright equilibrium position. The linearized models were implemented in MATLAB/Python and used for simulation, controller development and performance analysis. Several controllers were designed and implemented, including LQR, LQI and cascaded PID. In addition, an energy-based swing up controller was implemented to move the pendulum from the downward hanging position into a range where the balancing controllers could stabilize the system. All controllers were successfully implemented on the physical PLC-based setup and were capable of balancing the pendulum under normal operating conditions. The experimental results showed that the physical system generally behaved similarly to the simulated models, confirming that the developed mathematical models captured the dominant dynamics of the system. However, noticeable differences between simulation and physical behavior were observed during testing. These differences were primarily caused by unmodelled physical effects such as static friction, backlash, sensor noise, filtering delays actuator saturation and small mechanical imperfections in the setup. In particular, static friction in the conveyor system appeared to have a larger impact on the controller behavior than initially expected, especially for the controllers containing integral action. Among the tested controllers, the LQR controller provided smooth and stable balancing with relatively low oscillation while the LQI controller improved steady-state position regulation through the addition of integral action. The cascaded PID controller was also able to stabilize the pendulum, although it required substantial manual tuning compared to the state-space based controllers. The project furthermore demonstrated several practical implementation challenges related to real-time control systems including PLC cycle time limitations, encoder quantization, OPC UA communication delays and noisy velocity estimation. Overall, the project succesfully demonstrated both the theoretical and practical aspects of modelling and controlling an unstable dynamic system. The developed controllers achieved stable balancing of the inverted pendulum and the project provided valuable insight into dynamic modelling, simulation, controller implementation and real-world automation system integration.

References

[1] Schneider Electric, AC Servomotor BSH0552T01A2A, Last accessed 28.05.2026.
Available at: https://www.se.com/dk/da/product/BSH0552T01A2A/ac-servomotor-bsh-0-9-n-m-6000-rpm-uden-bremse-ip50/

[2] Schneider Electric, Lexium 32C Servo Drive, Last accessed 28.05.2026.
Available at: https://www.se.com/in/en/download/document/0198441113761-EN/

[3] B&R Industrial Automation, Programmable Logic Controller X20CP1382, Last accessed 28.05.2026.
Available at: https://bippa.ru/upload/iblock/522/ge8hmgplesp451nxbu1bmv42slwqn1yq/X20CP1382-en.pdf

[4] Cheng Fang, Notes: *System parameters*, March 2026
Available on ItsLearning

[5] Christoffer Sloth, *Modeling of Electromechanical Systems*, Lecture Notes: *Lektion 3. Bevægelse og kræfter*, June 2025
Available on ItsLearning

[6] Christoffer Sloth, *Modeling of Electromechanical Systems*, Lecture Notes: *Lektion 11: Euler-Lagragne modellering*, June 2025
Available on ItsLearning

[7] AsyncIO Documentation, OPC-UA Basics, Last accessed 28.05.2026.
Available at: https://opcua-asyncio.readthedocs.io/en/latest/usage/get-started/opc-ua-basics.html

[8] B&R Automation Studio, May 2026, As seen:

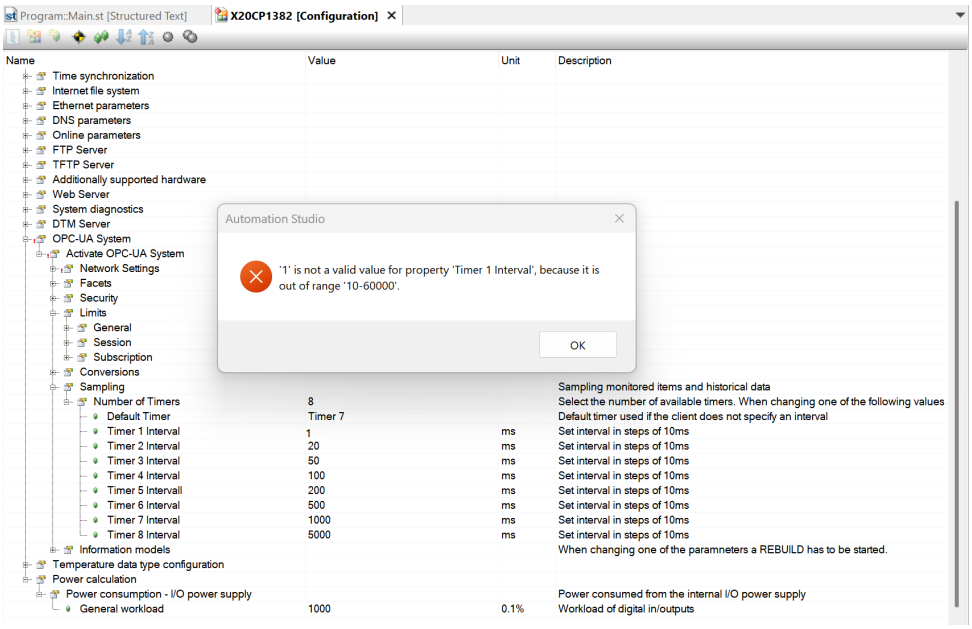

Figure 8.1: Automation studio showing 10ms being the lower limit

A Distribution of Tasks

Table A.1: Distribution of Tasks

Task	Noah	Rasmus	Emma	Jannick	Eymundur
Development					
System Modeling			x	x	x
Control Theory				x	x
PLC Implementation		x			
Data Collection	x				x
Report					
Abstract	x	x	x		
Introduction			x		
System Modeling			x		
Control Theory				x	
PLC Implementation		x			
Data Collection	x				x
Results	x	x			
Discussion		x		x	x
Conclusion			x		

B Code

The code is available in the attached ZIP file as well as on github at:
https://github.com/Noah091213/Semester-project-4-Inverted-pendulum

C Video Showcase

A video of the system running is available at:
https://youtube.com/shorts/rq7DjSa0-gI?feature=share